

1.5

```
ejercicio 1.5.py > ...
1  import pulp
2
3  # 1. Definir el problema (Minimización)
4  model = pulp.LpProblem("Cloud_Optimization", pulp.LpMaximize)
5
6  # 2. Definir Variables (Enteras, ya que no puedes rentar media instancia)
7  x1 = pulp.LpVariable("Backend", cat='Integer')
8  x2 = pulp.LpVariable("Dataworker", cat='Integer')
9
10 # 3. Función Objetivo
11 model += 300 * x1 + 250 * x2, "Costo_Total"
12
13 # 4. Restricciones
14 model += 2 * x1 + 1 * x2 <= 16, "RAM"
15 model += 1 * x1 + 2 * x2 <= 17, "SSD"
16 model += x1 <= 6, "Red"
17 model += x2 <= 7, "Licencias"
18
19 # 5. Resolver y mostrar
20 model.solve()
21
22 print(f"Estado: {pulp.LpStatus[model.status]}")
23 print(f"contenedor backend: {x1.varValue}")
24 print(f"contenedor data worker: {x2.varValue}")
25 print(f"Valor de Rendimiento Máximo: ${pulp.value(model.objective):.0f} USD/hora")
26
27
28 #source .venv/bin/activate
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g.

Option for printingOptions changed from normal to all
Total time (CPU seconds): 0.01 (Wallclock seconds): 0.01

Estado: Optimal
contenedor backend: 5.0
contenedor data worker: 6.0
Valor de Rendimiento Máximo: \$3000 USD/hora

[Done] exited with code=0 in 0.307 seconds

[Running] python -u "c:\Investigacion de operaciones\InvestigaciondeOPERACIONES\ejercicio 1.5.py"
Welcome to the CBC MILP Solver

1.6

```
Ejercicio 1.6.py > ...
1  import pulp
2
3  # 1. Definir el problema (Minimización)
4  model = pulp.LpProblem("Cloud_Optimization", pulp.LpMinimize)
5
6  # 2. Definir Variables (Enteras, ya que no puedes rentar media instancia)
7  x1 = pulp.LpVariable("Estandar",cat='Integer')
8  x2 = pulp.LpVariable("Premium",cat='Integer')
9
10 # 3. Función Objetivo
11 model += 20 * x1 + 60 * x2, "Costo_Total"
12
13 # 4. Restricciones
14 model += 1 * x1 + 3 * x2 >= 15, "Unidades de velocidad"
15 model += 2 * x1 + 2 * x2 >= 14, "Dias de retencion"
16
17 # 5. Resolver y mostrar
18 model.solve()
19
20 print(f"Estado: {pulp.LpStatus[model.status]}")
21 print(f"Contratar Tipo A: {x1.varValue}")
22 print(f"Contratar Tipo B: {x2.varValue}")
23 print(f"Costo Mínimo Diario: ${pulp.value(model.objective)}")
24
25 #source .venv/bin/activate
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Option for printingOptions changed from normal to all

Total time (CPU seconds): 0.01 (wallclock seconds): 0.00

Estado: Optimal

Contratar Tipo A: 3.0

Contratar Tipo B: 4.0

Costo Mínimo Diario: \$300.0

[Done] exited with code=0 in 0.312 seconds

1.7

```
Ejercicio 1.7.py > ...
1  import pulp
2
3  # 1. Definir el problema (Minimización)
4  model = pulp.LpProblem("Cloud_Optimization", pulp.LpMaximize)
5
6  # 2. Definir Variables (Enteras, ya que no puedes rentar media instancia)
7  x1 = pulp.LpVariable("Personaje",cat='Integer')
8  x2 = pulp.LpVariable("Escenario",cat='Integer')
9
10 # 3. Función Objetivo
11 model += 80 * x1 + 60 * x2, "Costo_Total"
12
13 # 4. Restricciones
14 model += 2 * x1 + 1 * x2 <= 12, "Tiempo GPU"
15 model += 1 * x1 + 2 * x2 <= 14, "Memoria de video"
16
17 # 5. Resolver y mostrar
18 model.solve()
19
20 print(f"Estado: {pulp.LpStatus[model.status]}")
21 print(f"Contratar Tipo A: {x1.varValue}")
22 print(f"Contratar Tipo B: {x2.varValue}")
23 print(f"Costo Mínimo Diario: ${pulp.value(model.objective)}")
24
25 #source .venv/bin/activate
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Option for printingOptions changed from normal to all

Total time (CPU seconds): 0.02 (Wallclock seconds): 0.02

Estado: Optimal

Contratar Tipo A: 4.0

Contratar Tipo B: 4.0

Costo Mínimo Diario: \$560.0

[Done] exited with code=0 in 0.355 seconds